INVERTED FILES & OTHER INDICES FOR TEXT

Inverted files · Suffix trees and arrays · Signature files

Information Retrieval Systems · DLOC-IV · BE IT, Semester VII

Why Not Just Scan the Text?

Scan every document for the query word (like grep)	**Look the word up in a prepared index, jump straight to the answers**
cost grows with the size of the collection	*cost depends mostly on the answer, not the collection*

- Sequential scanning is fine for small or fast-changing text. For a large, mostly static collection, you pay once to build an index and then every query is cheap.
- **An index is data about where things are, built in advance, so the search never has to read the text itself.**

Real life: the index at the back of this textbook. You never reread 700 pages to find 'stemming'; you look it up.

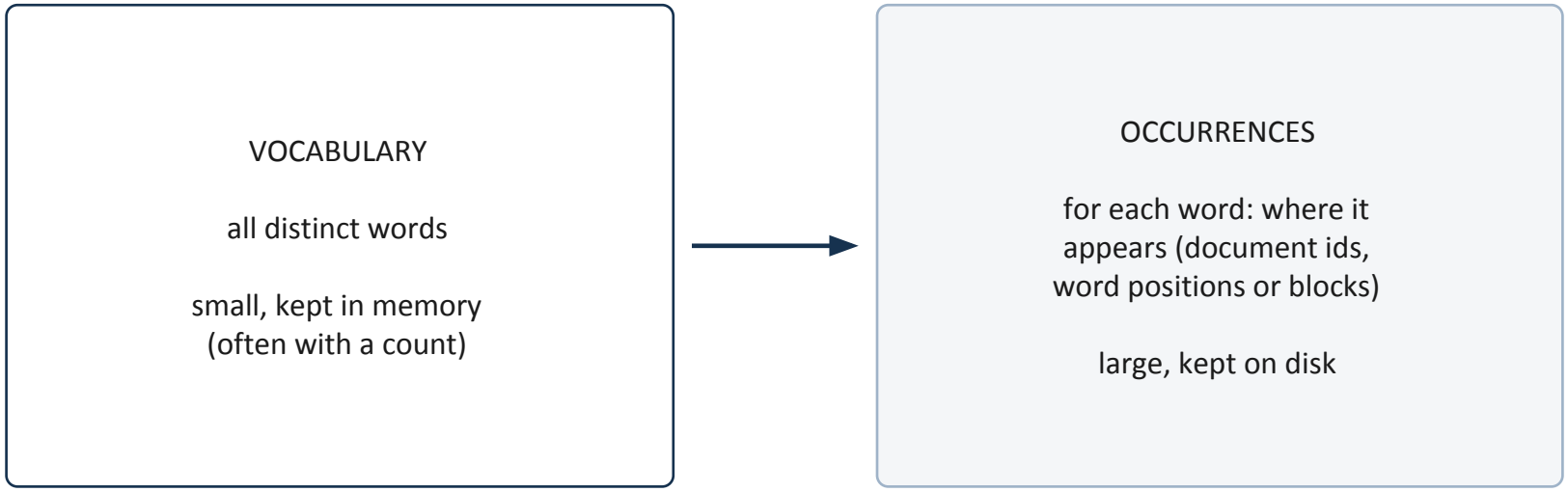
Three Index Families in This Lecture

Inverted files	Suffix trees and arrays	Signature files
word to list of places	index every text position	bit masks per text block
the workhorse of every search engine	any substring, no word boundaries needed	small, hash-based, may give false drops

- Inverted files are word-based. Suffix structures and signatures are the 'other indices for text' in the chapter.
- Each is a different trade between space, speed and what kinds of query it can answer.

Manning & Raghavan: Ch.1-5 cover inverted indexes in depth. Suffix and signature structures are Baeza-Yates material.

Inverted File: Structure



- 'Inverted' because the text maps document to words; the index flips it into word to documents.
- **The two parts have very different sizes, which drives every design decision that follows.**

Real life: the back-of-book index. Word = vocabulary entry, page numbers = occurrences.

Worked Example: Building the Index

Text (14 words):

That	house	has	a	garden	The	garden	has	many	flowers	The	flowers	are	beautiful
1	2	3	4	5	6	7	8	9	10	11	12	13	14

Vocabulary	a	are	beautiful	flowers	garden	has	house	many	that	the
Occurrences	4	13	14	10, 12	5, 7	3, 8	2	9	1	6, 11

- Walk the text left to right, and each time you see a word append its position to that word's list.
- Sort the vocabulary alphabetically so it can be searched fast.
- **This is a word-position index. Positions let us answer phrase and proximity queries later.**

The book's own example text, 'That house has a garden. The garden has many flowers. The flowers are beautiful', indexed by word number.

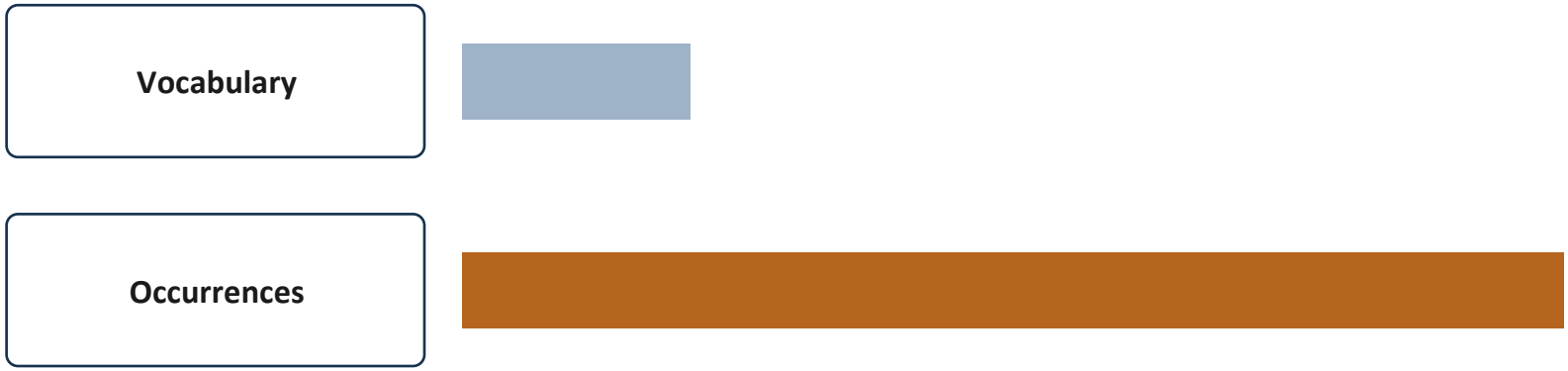
How Precisely Should We Record 'Where'?

Granularity	What an occurrence stores	Index size	What it can answer
Word position	exact position in the text	largest	phrases, proximity, exact location
Document	only which documents contain the word	smaller	Boolean and ranked document retrieval
Block	which fixed-size block of text	smallest	needs a scan of the matching blocks to confirm

- Block addressing groups the text into blocks (possibly several documents, or part of one) and records only block numbers.
- It shrinks the index a lot, at the cost of scanning the candidate blocks at query time to find the exact match.
- The book's headline result: with block addressing, an index of a few percent of the text size is possible, versus a much larger full-position index.

Real life: a book index that cites page numbers (block) rather than line numbers (exact position).

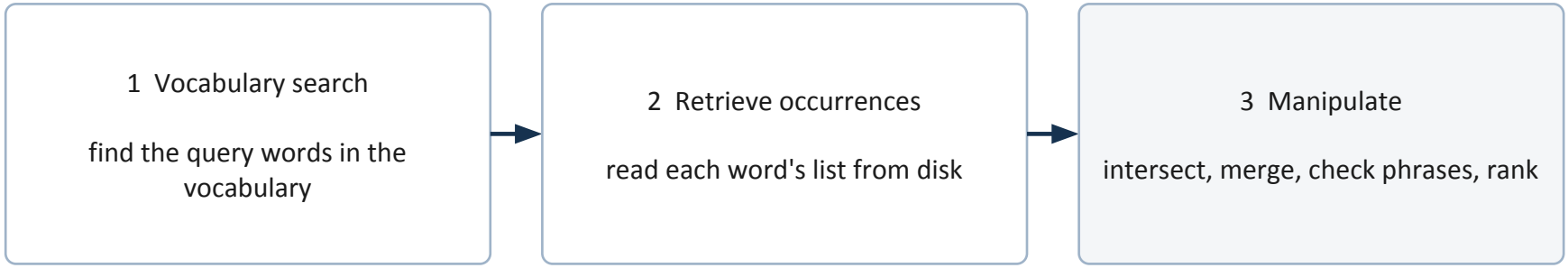
Vocabulary Is Small, Occurrences Are Big



- Heaps' law: vocabulary size grows roughly as n to a power between about 0.4 and 0.6, where n is the text size. The vocabulary grows much more slowly than the text.
- So the vocabulary fits in memory even for very large collections, while the occurrence lists dominate disk space.
- **Design rule: search the small vocabulary in memory, then do one disk access for the matching occurrence list.**

Real life: a 10-million-page collection still has a vocabulary of only a few million distinct terms.

Searching an Inverted File



- Steps 1 and 2 are often cheap; step 3 can dominate when lists are long.
- Multi-word queries are processed shortest list first, so intermediate results stay small.
- **Queries with one word need only steps 1 and 2.**

Real life: a search for 'python tutorial' reads two lists, then intersects them.

Worked Example: Answering Queries

`garden: {5, 7}` `flowers: {10, 12}` `the: {6, 11}` `has: {3, 8}`

Query: garden AND flowers

both words occur, so the text matches
(positions 5, 7 and 10, 12)

Phrase: "the flowers"

need flowers at the position right after
the: the=11, flowers=12 yes

Phrase: "garden has"

garden=7, has=8: consecutive, so yes
(garden=5 is followed by 'the', so no)

Proximity: garden within 3 words of flowers

7 and 10 differ by 3 yes

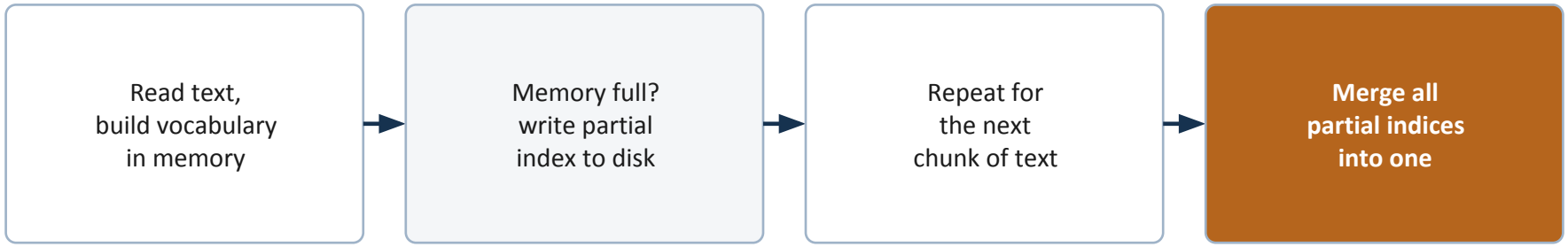
Positions turn phrase and proximity queries into simple arithmetic on the occurrence lists.

How to Search the Vocabulary Fast

Structure	Idea	Good at	Weakness
Hash table	hash the word to a slot	exact word lookup, very fast	no prefix or range queries
Trie	tree with one letter per edge	exact and prefix search	more memory
B-tree / sorted array	keep words in alphabetical order	ranges and prefixes, disk friendly	slower than hashing for exact match

Real life: autocomplete needs prefix lookup, so search boxes lean on trie or sorted-order structures.

Constructing an Inverted File



- The text is processed in chunks that fit in memory; each chunk produces a small sorted partial index.
- A final merge combines the partial lists, so the cost is roughly proportional to the amount of text.
- **Manning's Ch.4 gives two named versions of the same idea: BSBI (blocked sort-based) and SPIMI (single-pass in-memory), plus MapReduce for huge collections.**

Real life: Google, Lucene and Elasticsearch all build segments and merge them in the background.

Inverted Files in the Wild

Search engines	Lucene, Elasticsearch, Solr	Your phone
Web search is built on inverted indexes over billions of pages	the engines behind site search, log search, e-commerce	Spotlight / file search finds a word in thousands of files instantly

- Same three ideas every time: a small vocabulary, large occurrence lists, and an intersection step at query time.
- What changes is scale: compression, distribution across machines, and keeping the index fresh.

Manning Ch.5 (compression) and Ch.4 (distributed indexing) cover how these ideas scale.

Other Indices for Text

- An inverted file indexes words. What if the text has no clear words, or the query is an arbitrary substring?
- **DNA is one long string of A, C, G, T. Chinese and Thai text is written without spaces. A query like 'find GATTACA' or 'any occurrence of the substring ack' is not a word query.**
- Two families answer this: suffix trees and arrays (index every position), and signature files (a compact hash-based filter).

Real life: genome databases, compressed text search, and plagiarism detection all need substring indexes.

Suffix Trees: Index Every Suffix

Text: b a n a n a (positions 1 to 6)

Start	Suffix
1	banana
2	anana
3	nana
4	ana
5	na
6	a

- A suffix is the text from some position to the end.
- A suffix tree is a trie of all suffixes, with single-child chains compressed (a Patricia tree). Each leaf stores the start position.
- **Every substring of the text is a prefix of some suffix, so any substring is a path from the root.**

Searching for a pattern of length m takes about m steps, independent of the text size.

Suffix Arrays: The Compact Version

Sort the suffixes alphabetically, and keep only their start positions.

Rank	1	2	3	4	5	6
Suffix array	6	4	2	1	5	3
Suffix it points to	a	ana	anana	banana	na	nana

- The array is just n integers: a sorted list of pointers into the text. The text is not copied.
- Because the suffixes are sorted, all suffixes that start with the same pattern sit next to each other.
- **That adjacency is the whole trick: a pattern match becomes a contiguous range in the array.**

Real life: this is the structure behind many genome aligners (their FM-index is a compressed relative).

Searching a Suffix Array

Query: "ana"

6	a	*ana > a go right*
4	**ana**	*match*
2	**anana**	*match*
1	banana	*ana < banana stop*
5	na	
3	nana	

- Binary search finds the first and last suffix that begin with the pattern.
- The range is the answer: ranks 2 and 3 give positions 4 and 2, so 'ana' occurs at positions 2 and 4 (and overlaps itself).
- **Cost: about m log n character comparisons, for pattern length m and text length n.**

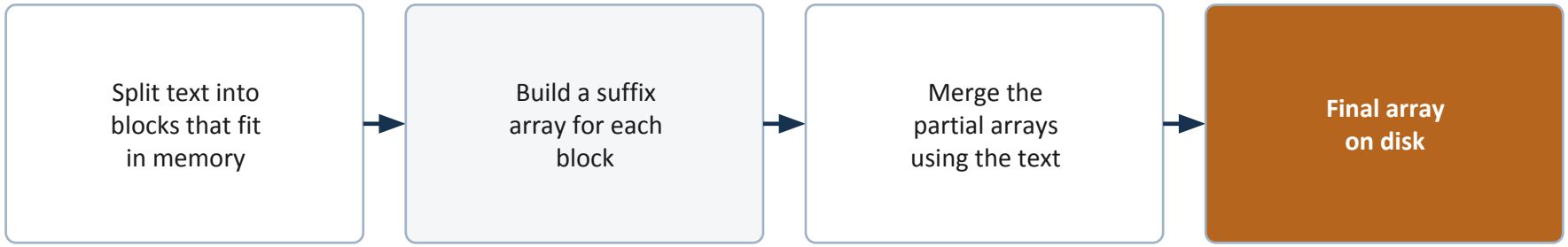
Note the overlap: 'banana' contains 'ana' twice, sharing the middle 'a'. An index finds both.

Suffix Tree vs Suffix Array

	Suffix tree	Suffix array
Search time	about m (pattern length)	about m log n
Space	much larger (pointers per node)	n integers, far smaller
Substring and prefix queries	yes	yes
Typical use	when speed matters most and memory is available	large texts; the practical default

- **Both let you search any substring, which an inverted file cannot do without extra structures.**
- Both index all positions, so the index can be larger than a word-level inverted file unless you index only chosen 'index points'.

Building a Suffix Array for a Large Text



- Sorting suffixes means comparing text far beyond the current block, so merging needs access to the text on disk.
- The book's section on large texts is about doing this merge efficiently with limited memory.
- **Same pattern as inverted-file construction: build in memory, spill partial results, merge.**

Suffix Structures in the Wild

Genomics	East Asian text	Plagiarism and repeats
locate a short DNA read inside a 3-billion-letter genome	Chinese, Japanese and Thai have no spaces, so word indexes are hard	find the longest string shared by two documents

- What these share: the query is an arbitrary string, not a dictionary word.
- Real genome tools use compressed relatives of the suffix array (the FM-index) to keep memory small.

Signature Files: Structure

- A word-oriented index built from hashing, with one fixed-size bit mask per block of text.
- **Each word hashes to a bit mask with a few bits set. A block's signature is the OR of the masks of all its words.**

Word	cat	dog	fish	bird	emu
Mask (8 bits)	10010000	01001000	00100100	00010001	11000000

Block	Words	Signature (OR of masks)
1	cat, dog	11011000
2	fish, bird	00110101

Masks are tiny and illustrative; real signatures use hundreds of bits per block.

Searching: Candidates and False Drops

Query word	Its mask	Block 1 (11011000)	Block 2 (00110101)
cat	10010000	all bits present: candidate	bit 1 missing: rejected
fish	00100100	bit 3 missing: rejected	all bits present: candidate
emu	11000000	all bits present: FALSE DROP	bit 1 missing: rejected

- A block is a candidate if every bit of the query mask is set in its signature.
- A block that fails a bit is definitely a non-match. A block that passes might still be wrong: that is a false drop.
- **Every candidate block is then scanned sequentially to confirm. 'emu' passes block 1 by accident, and the scan discards it.**

No false negatives, only false positives. Block 1 never contained 'emu'; its bits just happened to cover the mask.

Boolean Queries, Costs and Modern Cousins

- **Boolean queries.**
  - AND: the block must contain all word masks. OR: any one mask. Both are simple bit tests per signature.
- **Strengths.**
  - Very small index; fast to build and easy to update; simple.
- **Weaknesses.**
  - Search is linear in the number of signatures; false drops cost extra scanning; weak for phrases and proximity.
- **Verdict in the book:**
  - inverted files generally win for large collections; signature files suit small collections or tight space.

Modern cousin: the Bloom filter, a hash-based 'maybe present' test. Used in browsers, databases such as Cassandra, and Microsoft's Bing BitFunnel signature-style index.

Choosing an Index

	Inverted file	Suffix array / tree	Signature file
Indexes	words	every text position	hash of words per block
Query types	words, Boolean, phrase	any substring or prefix	words, Boolean
Index size	moderate (small with blocks)	large	smallest
Search cost	very fast	fast (log n)	linear in signatures
Best for	web and document search	DNA, no-space languages	small or space-limited

No single winner: the right index depends on what you must be able to ask.

Exam Tips

1. Inverted file: define vocabulary and occurrences, name the three addressing choices (word, document, block), and state searching in three steps.

2. Always back an answer with a small worked example: the 'garden' text for inverted files, 'banana' for suffix arrays, the 8-bit masks for signatures.

3. Signature files: explain false drops (no false negatives) and why inverted files usually beat them.

One Closing Thought

Every fast search you have ever done was answered before you typed it. The index is where the thinking happened; the query just reads the result.